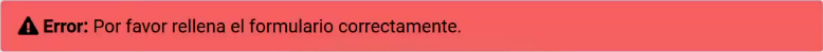


VALIDACIÓN DE FORMULARIOS DESDE HTML Y JAVA SCRIPT

Por el profe. JaiMeza², documento actualizado el 9 de junio del 2026, a las 8:00 AM

Usuario	Nombre
carlos 	John Doe
Contraseña	Repetir Contraseña
Correo Electrónico	Teléfono
john@correo.com	4491234567
☐ Acepto los Terminos y Condiciones	
 Error: Por favor rellena el formulario correctamente.	
 Enviar	

VALIDACIÓN DE LOS CAMPOS DEL FORMULARIO CON HTML

1.- ATRIBUTO PATRÓN: PATTERN (CLASE TEXTBOX-TEXT)

El atributo PATTERN se utiliza dentro de la etiqueta INPUT para especificar una expresión regular (RegEx Regular Expression) esto es que reglas del valor del campo de entrada se deben cumplir para que dato ingresado sea válido.

DÓNDE SE UTILIZA PATTERN

El atributo PATTERN también se aplica como atributo a los elementos:

INPUT: en tipos de texto (text), búsqueda (search), URL, teléfono (tel), correo electrónico (email) y contraseña (password).

CÓMO FUNCIONA Y REGLAS CLAVE:

Una expresión regular es una cadena de caracteres (string) que sigue un formato específico y que indica cual es la regla o patrón que debe seguir otra String de comparación, para determinar si cumple o no el patrón dado, se usan las expresiones regulares (RegEx) para evitar crear funciones en Java Script

Si el texto ingresado en el campo de entrada no coincide con el patrón codificado en la expresión regular (RegEx), el navegador bloquea el envío o no deja ingresar lo registrado y luego muestra un mensaje de error y si está presente el atributo TITLE adiciona el contenido del valor del título al mensaje de error.

Validación en el front-end: Si se incluye el atributo PATTERN este actúa como una función de validación en el navegador del cliente. Un hacker malintencionado puede saltarse esta regla fácilmente modificando el código de la página. Por lo tanto, **siempre se debe duplicar la validación en el servidor back-end**

1.1.- VALOR: "expresión regular"

"expresión regular" es el patrón RegEx

EJEMPLO DE CODIFICACIÓN

A continuación, se muestra los pares clave-valor prácticos de patrones comunes de Regular Expression

Nombre alfabético simple de usuario: Solo letras mayúsculas desde la ‘A’ a la ‘Z’, con una longitud de 8 caracteres, el par clave-valor es

```
pattern="[A-Z]{8}"
```

Nombre alfanumérico simple de usuario: Solo letras minúsculas desde la ‘a’ a la ‘z’ y números entre el ‘0’ y el ‘9’, con una longitud o rango entre 3 y 8 caracteres, el par clave-valor es

```
pattern="[a-z0-9]{3,8}"
```

Números de teléfono celulares en Colombia: Formato de 10 dígitos continuos (Ej. 3187935010), el par clave-valor es

```
pattern="\d{10}"
```

Contraseñas de baja seguridad: Mínimo 8 caracteres, incluyendo al menos una letra mayúscula, una letra minúscula y un número, el par clave-valor es

```
pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
```

Un ejemplo completo dentro del elemento FORM se codifica de la siguiente manera:

HTML SOURCE CODE: File name ‘index.html’

```
<!DOCTYPE html>
<html lang="es">
<!-- Formulario:Text-Input V_01 -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario:</title>
</head>

<body>
  <h3>Validación: INPUT/RegEx Regular Expression</h3>

  <form id="formulario_n01" name="formularioN01">
    <p>Ingrese su primer nombre:</p>
    <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" title="Nombre en mayúsculas y mínimo 3 letras"
      placeholder="Escriba acá su primer nombre" size="35" required="on" autocomplete="on" pattern="[A-Z]{3}">

    <p>Oprima el siguiente botón para enviar el formulario</p>
    <button type="submit">Enviar formulario</button>
```

```
</form>
</body>

</html>
```

PAGE PREVIEW: Live Server

Formulario: TextBox

127.0.0.1:5500/index.html?

Campo Lineal de Entrada de Texto: INPUT/RegEx Regular Expression

Ingrese su primer nombre:

 Utiliza un formato que coincida con el solicitado
Nombre en mayúsculas y mínimo 3 letras

<https://www.youtube.com/watch?v=wfogZfIS03U>

Guía Fácil y Completa de Expresiones Regulares desde Cero
FalconMasters

Carlos • Arturo →
449 • 123 • 45 • 67
carlos_@hotmail.com
www.carlos.com
Manuel • Alejandro
448-234-56-78
alejandror@outlook.com
https://www.manuel.alejandror.com.mx
http://alejandror.com.mx
Cesar • Alan
449 • 345 • 67 • 89

VALIDACIÓN DE LOS CAMPOS DEL FORMULARIO CON JS

Cuando el usuario hace clic en el botón de envío, si el valor del atributo ACTION (PROCESO) del elemento FORM (FORMULARIO) es una URL o ruta de destino entonces a esa dirección se enviarán los datos recopilados por un formulario, pero si el valor de action es "" ó "#" entonces JavaScript puede manejar todo el envío del formulario

CÓMO FUNCIONA Y REGLAS CLAVE

Para que JS pueda administrar el envío del formulario se debe hacer una copia del formulario, por ejemplo, en el ID `formulario_1` como

```
const formulario_1 = document.getElementById("formulario_n01");
```

Luego se debe adicionar el elemento del formulario_1 a la escucha de eventos de hardware externo (clic del ratón asociado al evento de envío 'submit' de formulario) además de la función `validar_campos`.

Nota: no se le coloca () porque de lo contrario se lanza o ejecuta la función, esto es, en base a la sentencia anterior

```
formulario_1.addEventListener("submit", validar_campos);
```

SINTAXIS DEL MÉTODO `addEventListener`

```
element.addEventListener(evento-type, function, options);
```

Adjunta una función de controlador de eventos a un elemento DOM específico sin sobrescribir los controladores de eventos existentes. Permite que JavaScript "escuche" las interacciones del usuario, como clics, pulsaciones de teclas o movimientos del ratón, y ejecuta la función inmediatamente cuando se produzcan esos eventos.

PARAMENTROS

EVENT-TYPE: Una cadena que especifica el nombre del evento (por ejemplo, 'click', 'keydown', 'submit'). No utilice el prefijo "on" (use 'click', no 'onclick').

FUNCTION: Es la función de devolución de llamada que se ejecuta cuando se activa el evento.

Notas

Si el argumento de función se pasa con el nombre de la función debe ir sin argumentos

Si el argumento es una definición de función puede tener paréntesis

OPTIONS (OPCIONAL): Un objeto que especifica características (como { once: true } o { passive: true }), o un valor booleano que representa useCapture

Tipos de eventos comunes

Eventos del ratón/Mouse Events: 'click', 'dblclick', 'mouseover', 'mouseout'.

Eventos del teclado/Keyboard Events: 'keydown', 'keyup', 'keypress'.

Eventos del formulario/Form Events: 'submit', 'change', 'input'.

Eventos de ventana/documento/Window/Document Events: 'load', 'resize',

DETECCIÓN DEL EVENTO DE INTERRUPCIÓN DE ESCRITURA (PULSACIÓN DE TECLA) SOBRE UN CAMPO DE ENTRADA

DETERMINANDO EL NOMBRE DE UN CAMPO DE EDICIÓN

Ejercicio: para una determina entrada de un formulario, mostrar el nombre de la entrada cada vez que se termine de pulsar una tecla de escritura de un dicho campo de entrada.

HTML SOURCE CODE: File name 'index.html'

```
<!DOCTYPE html>
<html lang="es">
<!-- Formulario:Text-Input V_0.1 -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario</title>
</head>

<body>
  <h3>Formulario: FORM/addEventListener:submit</h3>

  <form id="formulario_n01" name="formularioN01" action="#">
    <p>Ingrese su primer nombre:</p>
    <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" placeholder="Escriba acá su primer nombre"
      size="35" value="Luisa" required="on" autocomplete="off">

    <p>Ingrese su primer apellido:</p>
    <input type="text" id="caja_entrada_n02" name="CajaEntradaN02" placeholder="Escriba acá su primer apellido">

    <p>Oprima el siguiente botón para enviar el formulario</p>
    <input type="submit" id="boton_envio_n01" name="botonEnvioN01" value="Enviar formulario">
  </form>

  <script src="procesar_formulario_1.js">

</script>

</body>
```

```
</html>
```

JS SOURCE CODE: File name 'procesar_formulario_1.js'

```
//detección del evento de interrupción de escritura (pulsación de tecla) sobre un campo de entrada
```

```
// Arreglo de todos los elementos tipo input de "formulario_n01 input"
```

```
const entradas_form1 = document.querySelectorAll("#formulario_n01 input");
```

```
// entradas_form1 -> { "caja_entrada_n01", "caja_entrada_n02" }
```

```
function adminisitar_entrada(evento_escritura) {
```

```
  let nombre_entrada = evento_escritura.target.name;
```

```
  console.log(
```

```
    "El usuario escribió en el Campo de entrada llamado: " + nombre_entrada,
```

```
  );
```

```
}
```

```
// Adicionar a cada entrada del arreglo entradas_form1 el evento addEventListener:"keyup" (Tecla arriba)
```

```
entradas_form1.forEach((entrada_i) => {
```

```
  entrada_i.addEventListener("keyup", adminisitar_entrada);
```

```
});
```

PAGE PREVIEW: Live Server

The screenshot shows a web browser window with a single tab titled 'Formulario'. The address bar shows the URL '127.0.0.1:5500/index.html'. The page content includes a form with two text input fields. The first field is labeled 'Ingrese su primer nombre:' and contains the text 'Claudia'. The second field is labeled 'Ingrese su primer apellido:' and contains the text 'Juan'. Below the fields is a button labeled 'Enviar formulario'. At the bottom of the browser window, the developer console is open, showing two log messages: 'El usuario escribió en el Campo de entrada llamado: CajaEntradaN02' and 'El usuario escribió en el Campo de entrada llamado: CajaEntradaN01'. The messages are highlighted with a red box.

DETERMINANDO EL VALOR DE UN CAMPO DE EDICIÓN

Ejercicio Igual al anterior, pero adicionalmente se debe mostrar el valor de la entrada, asociada solo al primer campo de entrada

HTML SOURCE CODE: File name 'index.html'

```
<!DOCTYPE html>
<html lang="es">
<!-- Formulario:Text-Input V_0.2 -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario</title>
</head>

<body>
  <h3>Formulario: FORM/addEventListener</h3>

  <form id="formulario_n01" name="formularioN01" action="#">
    <p>Ingrese su primer nombre:</p>
    <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" placeholder="Escriba acá su primer
nombre"
      size="35" required="on" autocomplete="off">

    <p>Ingrese su primer apellido:</p>
    <input type="text" id="caja_entrada_n02" name="CajaEntradaN02" placeholder="Escriba acá su primer
apellido">

    <p>Oprima el siguiente botón para enviar el formulario</p>
    <input type="submit" id="boton_envio_n01" name="botonEnvioN01" value="Enviar formulario">
  </form>

  <script src="procesar_formulario_1.js">

  </script>

</body>

</html>
```

JS SOURCE CODE: File name 'procesar_formulario_1.js'

```
//detección del evento de interrupción de escritura (pulsación de tecla) sobre un campo de entrada

// Consultar el contenido del elemento asociado al identificador "caja_entrada_n01"
// y guardar una copia de este en el IdN caja_entrada1
const cajaEntrada1 = document.querySelector("#caja_entrada_n01");

function adminisitar_cajaEntrada1(evento_escritura) {
  // 'evento_escritura.target.value' lee el valor de cajaEntrada1 (lo que el usuario escribe).
  let valor_entrada = evento_escritura.target.value;
  let nombre_entrada = evento_escritura.target.name;
  console.log(
    "El usuario escribió:",
    valor_entrada,
    "en el campo de entrada",
    nombre_entrada,
  );
}

// Adicionar el elemento de la caja_entrada1 a la escucha de eventos de hardware externo (Input/campo de
entrada de escritura de texto) y luego ejecutar la función administrador_cajaEntrada1.
// Nota: no se le coloca () porque de lo contrario se lanza o ejecuta la función

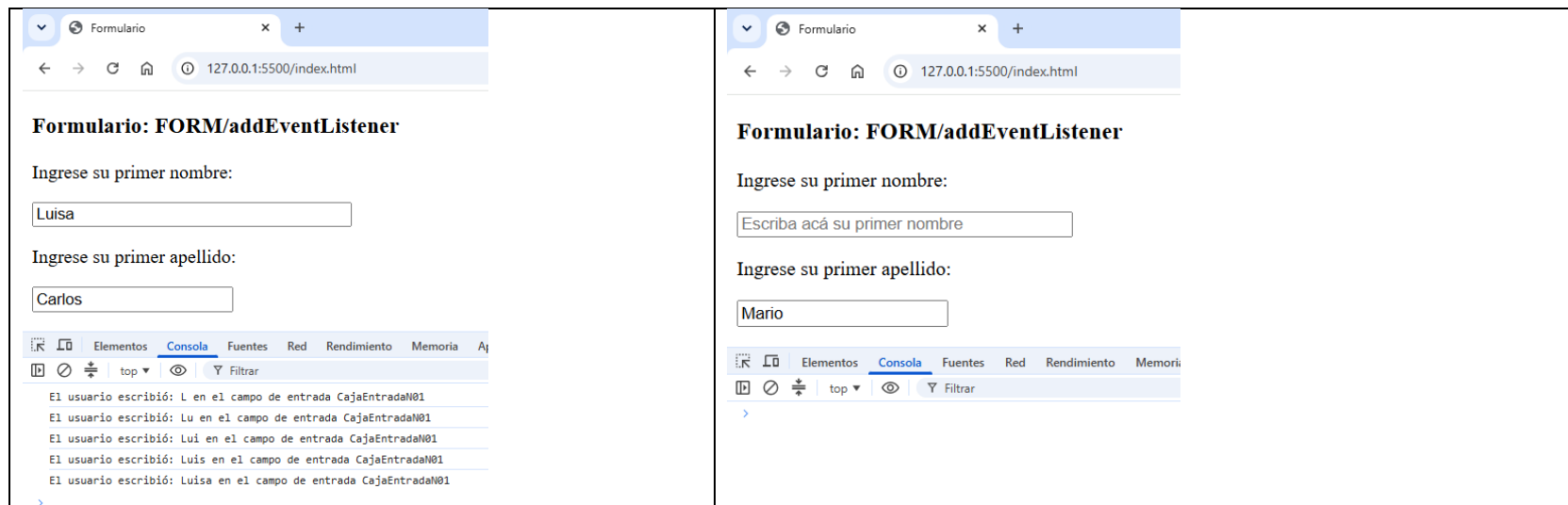
cajaEntrada1.addEventListener("input", adminisitar_cajaEntrada1);
// <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" placeholder="Escriba acá su primer nom-
bre">

// const botonEnvio1 = document.querySelector('#boton_envio_n01');
// function hola(evento_escritura) {
//   evento_escritura.preventDefault();
//   console.log("El holi:");
// }
// botonEnvio1.addEventListener('submit', hola);
```

PAGE PREVIEW: Live Server

Primer campo de entrada

Segundo campo de entrada



Nota: Al pulsar el botón "Enviar formulario", la barra de la URL debe cambiar

PREVENIR ACCIÓN DE ENVÍO

Para que JavaScript prevenga el envío automático de los datos del formulario por parte del navegador y así evitar que la página se refresque por la acción de enviar, se debe usar la CANCELACIÓN DE ACCIÓN PREDETERMINADA DE ENVÍO con el método **preventDefault**

SINTAXIS DEL MÉTODO addEventListener

```
evento-name.preventDefault();
```

evento-name: Nombre del evento dado por el programador

CÓMO FUNCIONA Y REGLAS CLAVE

Envío de formularios: **preventDefault()** Impide que un elemento FORM recargue la página web o se redirija. Esto permite procesar los datos de entrada con JavaScript o enviarlos de forma asíncrona mediante fetch().

Navegación de enlaces: Impide que una etiqueta de anclaje <a> navegue a una URL diferente al hacer clic. Esto se usa ampliamente en aplicaciones de una sola página (SPA) para manejar el enrutamiento dinámicamente.

Atajos de teclado: Intercepta las pulsaciones de teclas como la barra espaciadora o las teclas de flecha para evitar que desplacen la página web, lo cual es muy útil en videojuegos basados en navegador.

Menús contextuales: Impide que aparezca el menú contextual predeterminado del clic derecho, lo que le permite crear menús contextuales personalizados.

Un ejemplo completo dentro del elemento FORM se codifica de la siguiente manera:

HTML SOURCE CODE: File name 'index.html'

```
<!DOCTYPE html>
<html lang="es">
<!-- Formulario:Text-Input V_0.3 -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario</title>
</head>

<body>
  <h3>Formulario: FORM/preventDefault()</h3>
```

```

    <form id="formulario_n01" name="formularioN01" action="#">
        <p>Ingrese su primer nombre:</p>
        <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" placeholder="Escriba acá su primer
nombre"
            size="35" value="Luisa" required="on" autocomplete="off">

        <p>Ingrese su primer apellido:</p>
        <input type="text" id="caja_entrada_n02" name="CajaEntradaN02" placeholder="Escriba acá su primer
apellido">

        <p>Oprima el siguiente botón para enviar el formulario</p>
        <input type="submit" id="boton_envio_n01" name="botonEnvioN01" value="Enviar formulario">
    </form>

    <script src="procesar_formulario_1.js">

    </script>

</body>

</html>

```

JS SOURCE CODE: File name 'procesar_formulario_1.js'

```

// Obtener el contenido del elemento asociado al identificador "formulario_n01" y guardar una copia de
este en el IdN formulario_1
const formulario_1 = document.getElementById("formulario_n01");

// Adicionar el elemento del formulario_1 a la escucha de eventos de hardware externo (Input/Submit clic
del ratón) y luego ejecutar la función.
// Nota: no se le coloca () porque de lo contrario se lanza o ejecuta la función

formulario_1.addEventListener("submit", function (evento_enviar) {
    // Evita que la página se recargue y los datos se envíen de forma tradicional
    evento_enviar.preventDefault();
    // Aquí puedes ejecutar la lógica, para enviar o validar datos
    console.log(
        "Formulario interceptado. Los datos NO se han enviado Procesando datos con JS...",
    );
});
// formulario_1.addEventListener("submit", (evento_enviar) => evento_enviar.preventDefault()); // forma
compacta
// <input type="submit" value="Enviar formulario">

```

PAGE PREVIEW: Live Server	
	Al pulsar el botón ="Enviar formulario", la barra de la URL no debe cambiar

Ejemplo para detener evento de ir a un enlace

```
<a href="www.google.com">SAN GOOGLE</a>

const link = document.querySelector('a');

link.addEventListener('click', (evento_ir_a) => {
  // Impide que el navegador navegue a la URL
  evento_ir_a.preventDefault();

  console.log('Se interceptó el evento de clic en el enlace...');
});
```

Ejercicio para todas las entradas del formulario cada vez que se termine de pulsar una tecla de escritura de un campo de entrada. Mostrar el mensaje "Se terminó de pulsar la Tecla (Tecla Arriba)"

HTML SOURCE CODE: File name 'index.html'

```
<!DOCTYPE html>
<html lang="es">
<!-- Formulario:Text-Input V_0.4 -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario</title>
</head>

<body>
  <h3>Formulario: FORM/addEventListener:keyup</h3>

  <form id="formulario_n01" name="formularioN01" action="#">
    <p>Ingrese su primer nombre:</p>
    <input type="text" id="caja_entrada_n01" name="CajaEntradaN01" placeholder="Escriba acá su primer nombre"
      size="35" value="Luisa" required="on" autocomplete="off">

    <p>Ingrese su primer apellido:</p>
    <input type="text" id="caja_entrada_n02" name="CajaEntradaN02" placeholder="Escriba acá su primer apellido">

    <p>Oprima el siguiente botón para enviar el formulario</p>
    <input type="submit" id="boton_envio_n01" name="botonEnvioN01" value="Enviar formulario">
  </form>

  <script src="procesar_formulario_1.js">

  </script>

</body>
</html>
```

JS SOURCE CODE: File name 'procesar_formulario_1.js'

```
// Obtener el contenido del elemento asociado al identificador "formulario_n01" y guardar una copia de
este en el IdN formulario_1
const formulario_1 = document.getElementById("formulario_n01");

// Adicionar el elemento del formulario_1 a la escucha de eventos de hardware externo (Input/Submit clic
del ratón) y luego ejecutar la función.
```

```
// Nota: no se le coloca () porque de lo contrario se lanza o ejecuta la función

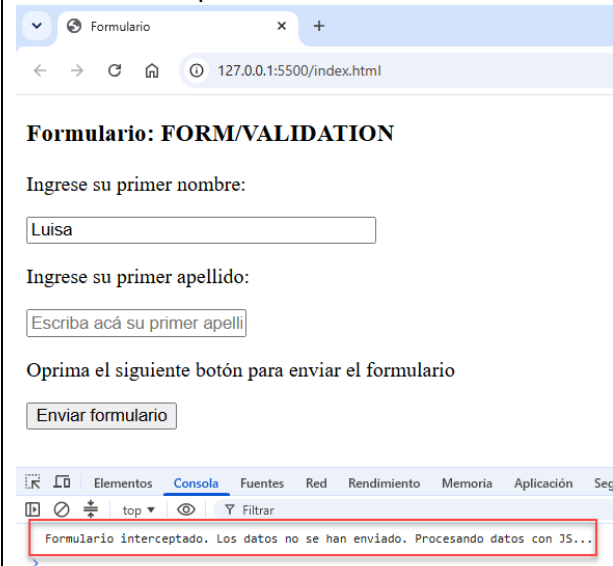
formulario_1.addEventListener("submit", function (evento_enviar) {
  // Evita que la página se recargue y los datos se envíen de forma tradicional
  evento_enviar.preventDefault();
  // Aquí puedes ejecutar la lógica, para enviar o validar datos
  console.log(
    "Formulario interceptado. Los datos NO se han enviado Procesando datos con JS...",
  );
});
// <input type="submit" value="Enviar formulario">

// Arreglo de todos los elementos tipo input de "formulario_n01 input"
const entradas_form1 = document.querySelectorAll("#formulario_n01 input");
// entradas_form1 -> { "caja_entrada_n01", "caja_entrada_n02" }

// Adicionar a cada entrada del arreglo entradas_form1 el evento addEventListener:"keyup" (Tecla arriba)
entradas_form1.forEach((entrada) => {
  entrada.addEventListener("keyup", () => {
    console.log("Se terminó de pulsar la Tecla (Tecla Arriba)");
  });
});
```

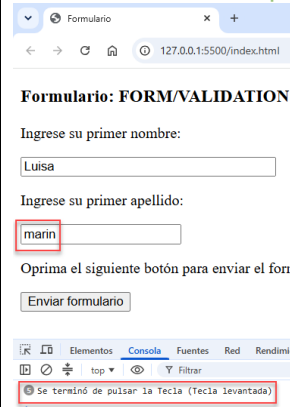
PAGE PREVIEW: Live Server

Sin editar campos de entrada



Editando campos de entrada: al digitar por ejemplo marin se realizan 5 eventos de pulsacion de tecla de ahi que se muestre

5 "Se terminó de pulsar la Tecla (Tecla levantada)"



Aprende a Validar Formularios con Javascript y Expresiones Regulares

FalconMasters <https://youtu.be/s3pC93LgP18>

